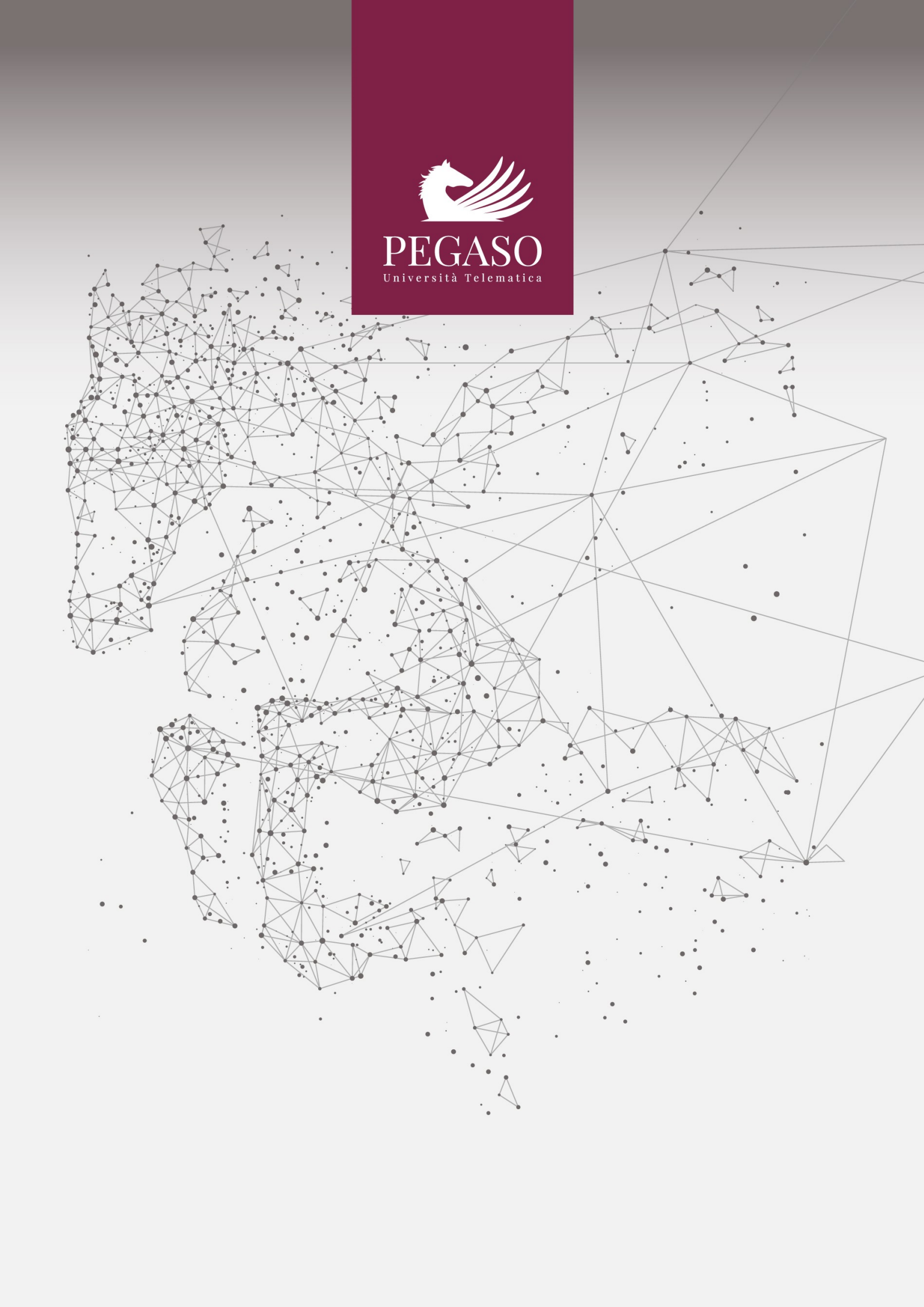

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. REACT NATIVE 3
2. DIFFERENZE TRA REACT E REACT NATIVE 5
3. COMPONENTI 6
4. EVENTI 10
5. STILI 12
6. COMPONENTI 15
BIBLIOGRAFIA 19

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. React native

React Native è un framework open source sviluppato da Facebook per la creazione di applicazioni mobili per iOS e Android utilizzando la sintassi di programmazione di React per il web. Grazie a React Native, gli sviluppatori possono creare applicazioni mobili che utilizzano componenti nativi come se fossero stati creati in modo nativo per la piattaforma specifica.

Una delle principali differenze tra React e React Native è il modo in cui vengono creati e gestiti i componenti. In React, i componenti sono creati utilizzando l'HTML e i CSS, mentre in React Native i componenti sono creati utilizzando componenti nativi come View, Text, Image, ecc.

La filosofia alla base di React Native è la stessa di React: la separazione tra lo stato e la presentazione, la creazione di componenti riutilizzabili e la gestione degli aggiornamenti del DOM in modo efficiente.

Ci sono diversi ambienti di sviluppo che si possono utilizzare per sviluppare applicazioni con React Native:

- **Expo**: è una piattaforma che semplifica lo sviluppo di app React Native, fornendo una serie di strumenti e servizi, tra cui un ambiente di sviluppo, una libreria di componenti UI predefiniti, un sistema di gestione dei pacchetti e molto altro ancora.
- **React Native CLI**: è uno strumento a riga di comando che ti consente di creare, gestire e compilare progetti React Native utilizzando le API native di iOS e Android.
- **Visual Studio Code**: è un editor di codice open-source che supporta lo sviluppo di app React Native. Puoi utilizzare diversi plugin e estensioni per integrare la sintassi React Native, la gestione dei pacchetti e il debugging all'interno dell'editor.
- **Android Studio**: è un ambiente di sviluppo integrato (IDE) di Google per lo sviluppo di app Android. Puoi utilizzare Android Studio per creare e gestire progetti React Native e compilare app Android.
- **Xcode**: è un IDE di Apple per lo sviluppo di app iOS. Puoi utilizzare Xcode per creare e gestire progetti React Native e compilare app iOS.

Soffermiamoci su Expo nella sua versione off-line ed on-line.

Per utilizzare Expo (off-line) per lo sviluppo di app React Native, è necessario installare Expo CLI (Command Line Interface) sul proprio computer: questo può essere fatto tramite npm (Node Package Manager), eseguendo il seguente comando nella finestra di terminale:

```
npm install -g expo-cli.
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Successivamente, si deve creare un nuovo progetto Expo utilizzando il comando:

```
expo init nome-del-tuo-progetto
```

Questo comando creerà una nuova directory per il progetto e installerà tutte le dipendenze necessarie.

Una volta creato il progetto, si può iniziare a sviluppare l'app utilizzando il proprio editor di codice preferito. Expo fornisce una serie di componenti UI predefiniti che possono essere utilizzati per costruire l'interfaccia utente, nonché una vasta gamma di API che consentono di accedere alle funzionalità native del dispositivo (come la fotocamera, il GPS, le notifiche push, ecc.).

Per avviare l'app, è necessario eseguire il comando:

```
expo start
```

Nella finestra di terminale. Questo comando aprirà Expo Dev Tools, un'interfaccia web che consente di avviare l'app sul dispositivo fisico o su un emulatore.

Infine, una volta completato lo sviluppo dell'app, è necessario distribuirla utilizzando Expo. Expo fornisce una serie di strumenti e servizi che consentono di pubblicare l'app su App Store e Google Play, nonché di condividere l'app con altre persone tramite link.

Se si desidera utilizzare la versione online di Expo, chiamata **Expo Snack**, è necessario seguire i seguenti passaggi per iniziare a sviluppare l'app React Native:

- **Accedere a Expo Snack**: si deve accedere al sito https://snack.expo.dev/ e accedere con il proprio account Expo o con il proprio account Google o GitHub.
- **Creare un nuovo progetto**: una volta effettuato l'accesso, si può creare un nuovo progetto Expo Snack selezionando "New Snack" nella parte superiore della pagina. Qui si può iniziare a scrivere il proprio codice React Native.
- **Aggiungere librerie**: Expo Snack fornisce un'ampia gamma di librerie React Native pre-installate che possono essere utilizzate per costruire l'app. Se si ha bisogno di librerie aggiuntive, è possibile aggiungerle utilizzando l'opzione "Add dependency" nel menu laterale.
- **Avviare l'app**: una volta che si è scritto il codice dell'app, si può avviarla utilizzando il pulsante "Run" nel menu laterale. L'app verrà compilata e avviata in un simulatore di dispositivo integrato in Expo Snack.
- **Condividere l'app**: una volta completato lo sviluppo dell'app, si deve condividerla con gli altri utilizzando l'opzione "Share" nel menu laterale. Questo genererà un link che si può inviare agli altri per permettere loro di visualizzare e testare l'app nel proprio browser.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Differenze tra react e react native

React e React Native sono entrambi framework sviluppati da Facebook per la creazione di interfacce utente. Tuttavia, ci sono alcune importanti differenze tra i due.

In React, i componenti sono creati utilizzando una combinazione di JavaScript e JSX, una sintassi simile a HTML. I componenti vengono poi renderizzati all'interno di un browser web utilizzando il DOM (Document Object Model).

In React Native, i componenti vengono creati utilizzando gli stessi concetti di base di React, ma vengono renderizzati tramite i componenti nativi di iOS e Android, anziché utilizzare il DOM. Ciò significa che i componenti React Native vengono compilati in codice nativo per il dispositivo in uso, anziché essere interpretati da un browser web.

Per quanto riguarda la gestione degli eventi, React utilizza i sintomi degli eventi DOM, come onClick, mentre React Native utilizza i gestori di eventi nativi per il sistema operativo in uso, come onPress per iOS e onPress per Android.

Per quanto riguarda lo styling, in React si utilizza una combinazione di CSS e inline styling, mentre in React Native si utilizza uno stile basato sulla sintassi di JavaScript, con alcune proprietà simili a quelle di CSS.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Componenti

In React, i componenti sono creati utilizzando una combinazione di JavaScript e JSX. Ad esempio, il seguente codice definisce un componente React che visualizza un elenco di elementi:

```
import React from 'react';

function ListItem(props) {
  return <li>{props.value}</li>;
}

function MyList(props) {
  const items = props.items.map((item) => <ListItem key={item.id}
value={item.value} />);
  return <ul>{items}</ul>;
}
```

In React Native, i componenti sono creati utilizzando gli stessi concetti di base di React, ma vengono renderizzati tramite i componenti nativi di iOS e Android. Ad esempio, il seguente codice definisce un componente React Native che visualizza un elenco di elementi:

```
// app.js
import React from 'react';
import { StyleSheet, View } from 'react-native';
import MyList from './MyList';

export default function App() {
  const items = [
    { id: 1, value: 'Primo elemento' },
    { id: 2, value: 'Secondo elemento' },
    { id: 3, value: 'Terzo elemento' },
  ];

  return (
    <View style={styles.container}>
      <MyList items={items} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    justifyContent: 'center',
  },
});

//MyList.js
import React from 'react';
import { FlatList, StyleSheet, Text, View } from 'react-native';

function MyList(props) {
  return (
    <View style={styles.container}>
      <FlatList
        data={props.items}
           renderItem={({item})=><Text
                style={styles.item}>{item.value}</Text>}
        keyExtractor={(item) => item.id.toString()}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  item: {
    padding: 10,
    fontSize: 18,
    height: 44,
  },
});

export default MyList;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tale codice presenta il problema della "barra di stato" in IOS:



Figura 1 *- Problema della barra di stato in IOS*

Per risolvere il problema si può operare nel seguente modo:

```
import React from 'react';
import { FlatList, StyleSheet, Text, View } from 'react-native';

function MyList(props) {
  const renderItem = ({ item }) => {
    const itemStyle = item.id === 1 ?
     [styles.item, styles.firstItem] : styles.item;
    return <Text style={itemStyle}>{item.value}</Text>;
  };

  return (
    <View style={styles.container}>
      <FlatList
        data={props.items}
        renderItem={renderItem}
        keyExtractor={(item) => item.id.toString()}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    alignItems: 'center',
    justifyContent: 'center',
    paddingTop: 20,
  },
  item: {
    padding: 10,
    fontSize: 18,
    height: 44,
  },
  firstItem: {
    marginTop: 20,
  },
});

export default MyList;
```

In questo modo, viene definita una funzione renderItem che applica uno stile aggiuntivo (styles.firstItem) solo al primo elemento della lista (item.id === 1). Lo stile styles.firstItem definisce un margine superiore di 20 pixel per spostare il primo elemento sotto la barra di stato.



Figura 2 *- Problema della barra di stato in IOS*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Eventi

In React, gli eventi vengono gestiti utilizzando la sintassi degli eventi DOM, come onClick o onChange. Ad esempio, il seguente codice definisce un componente React che gestisce il clic di un pulsante:

```
import React, { useState } from 'react';

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return <button onClick={handleClick}>You clicked me {count} times</button>;
}
```

In React Native, gli eventi vengono gestiti utilizzando i gestori di eventi nativi per il sistema operativo in uso, come onPress per iOS e onPress per Android. Ad esempio, il seguente codice definisce un componente React Native che gestisce il clic di un pulsante:

```
     import React, { useState } from 'react';
import { Button, StyleSheet, Text, View } from 'react-native';

export default function App() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  const styles = StyleSheet.create({
    container: {
      flex: 1,
      backgroundColor: '#fff',
      alignItems: 'center',
      justifyContent: 'center',
    },
  });

  return (
    <View style={styles.container}>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      <Button onPress={handleClick} title={`You clicked me ${count}
times`} />
    </View>
  );
}
```

Ecco la renderizzazione:

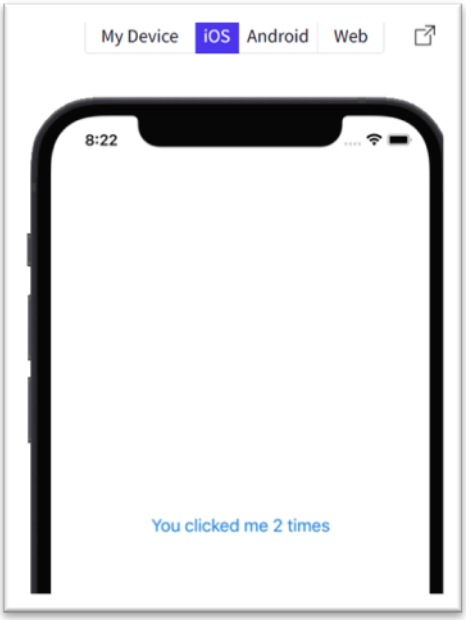


Figura 3 *- Eventi*

Nel codice abbiamo definito il componente principale dell'app App. Utilizziamo lo stato useState per tenere traccia del numero di volte che viene fatto clic sul pulsante: la funzione handleClick aggiorna lo stato incrementando il valore di count di 1 ogni volta che il pulsante viene premuto.

Utilizziamo lo stile StyleSheet.create per definire lo stile del contenitore dell'app; infine, restituiamo un componente View con il pulsante al suo interno. Il pulsante utilizza la funzione handleClick come evento onPress e visualizza il numero di volte che è stato premuto utilizzando la variabile count nel titolo del pulsante.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Stili

In React, gli stili possono essere definiti utilizzando una combinazione di CSS e inline styling. Ad esempio, il seguente codice definisce un componente React che visualizza del testo con uno stile specifico:

```
import React from 'react';

function MyText() {
  const style = {
    color: 'red',
    fontSize: '24px',
    fontWeight: 'bold',
  };

  return <p style={style}>Hello, World!</p>;
}
```

In React Native, gli stili vengono definiti utilizzando una sintassi simile a quella di JavaScript, con alcune proprietà simili a quelle di CSS. Ad esempio, il seguente codice definisce un componente React Native che visualizza del testo con uno stile specifico:

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

function MyText() {
  return <Text style={styles.text}>Hello, World!</Text>;
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  text: {
    color: 'red',
    fontSize: 24,
    fontWeight: 'bold',
  },
});

export default function App() {
  return (
    <View style={styles.container}>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      <MyText />
    </View>
  );
}
```

Ecco la renderizzazione:



Figura 4 *- Stile*

In questo blocco di codice, definiamo il componente MyText che restituisce il componente Text con il testo "Hello, World!" e lo stile definito nella costante styles.text.

Utilizziamo anche lo stile StyleSheet.create per definire lo stile del contenitore principale del componente App, styles.container. Impostiamo flex: 1 per far sì che il contenitore si estenda a tutta l'altezza della schermata, justifyContent: 'center' per allineare il contenuto al centro verticalmente e alignItems: 'center' per allineare il contenuto al centro orizzontalmente.

In React Native, gli stili vengono gestiti tramite i fogli di stile, che sono oggetti di JavaScript contenenti proprietà di stile. Le proprietà sono simili a quelle di CSS, ma sono scritte in camelCase invece che in kebab-case. Inoltre, le unità di misura sono specificate senza l'uso di stringhe (ad esempio, fontSize: 24 invece di fontSize: '24px').

Inoltre, React Native supporta la creazione di fogli di stile globali e locali. I fogli di stile globali possono essere definiti in un file separato e poi importati e utilizzati in tutta l'applicazione. I fogli di stile locali possono essere definiti direttamente nei componenti e vengono applicati solo a quel componente specifico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Camel case e **kebab case** sono due convenzioni di denominazione (naming convention) utilizzate per scrivere il codice. La differenza tra le due convenzioni sta nella scelta di come separare le parole all'interno del nome delle variabili o delle funzioni.

In camel case, la prima parola è in minuscolo e ogni parola successiva inizia con una lettera maiuscola, senza separazioni.

Ad esempio: firstName, totalAmount, checkUserName.

In kebab case, invece, ogni parola è separata dalle altre tramite un trattino (o dash) "-". Ad esempio: first-name, total-amount, check-user-name.

L'utilizzo di una convenzione di denominazione coerente è importante per migliorare la leggibilità e la manutenibilità del codice. In generale, il camel case viene utilizzato di più nel linguaggio di programmazione JavaScript e nel suo ecosistema (ad esempio, in React), mentre il kebab case viene utilizzato principalmente per la denominazione di classi CSS.

La convenzione di denominazione **camel case** prende il nome dalla forma del carattere: la parola inizia con una lettera minuscola, mentre ogni parola successiva inizia con una lettera maiuscola, creando un aspetto simile alla schiena di un cammello.

Ad esempio, la variabile firstName segue la convenzione camel case perché la prima parola inizia con una lettera minuscola e la seconda parola inizia con una lettera maiuscola.

La convenzione di denominazione **kebab case** prende il nome dalla forma del carattere trattino ("-"), che assomiglia alla forma di un kebab, un tipo di piatto mediorientale a base di carne grigliata o arrostita su uno spiedo. In particolare, il nome kebab case è stato coniato nel contesto di CSS, dove il kebab case viene spesso utilizzato per definire i nomi delle classi.

Inoltre, il kebab case è anche conosciuto con altri nomi, come **dash case** o **spinal case**.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Componenti

Come accennato in precedenza, in React Native i componenti sono nativi alla piattaforma, (come Text, View e Image…) anziché HTML come in React per il web: ciò significa che ci sono alcune differenze nella struttura dei componenti tra React e React Native.

In generale, è possibile fare una mappatura tra i tag HTML standard e i componenti nativi di React Native, ma è importante tenere in considerazione le differenze tra le due piattaforme e adattare di conseguenza la struttura dei componenti: non tutti i tag HTML standard hanno una corrispondenza diretta nei componenti nativi di React Native. Ad esempio, i tag form e input in React per il web non hanno una corrispondenza diretta in React Native, in quanto la gestione dei form e degli input è diversa sulla piattaforma mobile.

Ecco quella che potrebbe essere una "generica" mappatura:

tag HTML	Componente nativo React Native
div	View
span	Text
img	Image
button	TouchableOpacity
input	TextInput
form	Non disponibile in React Native
select	Picker
option	Picker.Item

Consideriamo ad esempio il componente nativo di React Native che viene utilizzato per rappresentare un div del web: View; tale componente è un contenitore flessibile e multiuso che viene utilizzato per raggruppare uno o più componenti all'interno di un'area di visualizzazione.

Ecco un esempio di utilizzo del componente View in React Native:

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const App = () => {
  return (
    <View style={styles.container}>
      <Text>Questo è un componente View di React Native!</Text>
    </View>
  );
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});

export default App;
```

In questo esempio, il componente View viene utilizzato come contenitore per il componente Text ed è stato configurato con uno stile attraverso l'utilizzo del metodo StyleSheet.create(). Questo metodo permette di definire uno o più stili CSS in un oggetto JavaScript, in modo da poterli applicare al componente.

Notare come il componente View abbia la proprietà flex: 1. Questa proprietà viene utilizzata per far espandere il componente View a tutta la dimensione dello schermo.

Ecco la renderizzazione in Expo Snack:



Figura 5 *- Expo Snack View*

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ecco un esempio di utilizzo del componente Text in React Native:

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const App = () => {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Questo è un componente Text di React Native!</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  text: {
    fontSize: 20,
    fontWeight: 'bold',
  },
});

export default App;
```

In questo esempio, il componente Text viene utilizzato per visualizzare una stringa di testo. Il componente Text è stato configurato con uno stile attraverso l'utilizzo del metodo StyleSheet.create(). In questo caso, il componente Text ha una dimensione del carattere di 20 punti e un peso del carattere in grassetto.

Notare come il componente Text sia contenuto all'interno del componente View. Il componente View viene utilizzato come contenitore generale e permette di definire uno stile globale per il contenuto dell'applicazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```jsx
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

const App = () => {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Questo è un componente Text di
React Native!</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
  text: {
    fontSize: 20,
    fontWeight: 'bold',
  },
});

export default App;
```



Figura 6 *- Expo Snack Text*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org
- REACT: https://it.legacy.reactjs.org/
- REACT NATIVE: https://reactnative.dev/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).